

Safe Edges

2025

Smart Contract Audit REPORT

ASSESSED ON OCT, 2025

The Riglabs

Security Assessment



SAFEEDGES.IN

Rigs lab Usdm Audit Report

Table of Contents

1. Executive Summary

2. Scope

3. Methodology

4. Severity Classification

5. Findings Overview

H-01 Inconsistent Timestamp Comparison Between Fuel TAI64 and Oracle UNIX Formats

M-01 Recursive transfer_ownership Call Causes Guaranteed Revert

L-01 get_price Uses Pyth Price Without Validating confidence Interval

I-01 Missing Zero Address Validation in transfer_ownership

I-02 get_price Always Prioritizes Stork Oracle, Ignoring Potentially Fresher Data

6. Detailed Findings (with code, impact, recommendations)

7. Conclusion

Executive Summary

Item	Details
Scope	https://github.com/Rig-Labs/Moor/blob/main/contracts/oracle-contract/src/main.sw
Timeframe	e.g., Sep 10–15, 2025

Methodology	Manual code review, static analysis, best practices check
Severity Summary	HIGH
Overall Risk	MEDIUM
Status	Fixed (choose one)

This audit of the RIG Labs USDM contracts reviewed core functionality and identified several issues affecting ownership, price verification, and prize settlement. Most findings can be fixed with straightforward code updates as recommended.

Techniques and Methods

Code quality checks

Best practices & documentation review

Gas efficiency analysis

Reentrancy & ERC compliance checks

Analysis Types

Structural Analysis – Reviewed design patterns & contract structure

Static Analysis – Automated scanning for vulnerabilities

Manual Analysis – Line-by-line code review

Gas Consumption – Optimization analysis

Tools: Remix IDE, Foundry, Slither, Mythril, Solhint

Checked Vulnerabilities

	Access Management		Compiler version not fixed
	Arbitrary write to storage		Address hardcoded
	Centralization of control		Divide before multiply
	Ether theft		Integer overflow/underflow
	Improper or missing events		ERC compliance
	Logical issues and flaws		Dangerous strict equalities
	Arithmetic Computations Correctness		Tautology or contradiction
	Race conditions/front running		LOW
	SWC Registry		Missing Zero Address Validation
	Re-entrancy		Private modifier
	Timestamp Dependence		Revert/require functions
	GAS		Multiple Sends
	Exception Disorder		Using suicide
	Gasless Send		Using delegatecall
	Use of tx.origin		Upgradeable safety

Malicious libraries		Using throw
---------------------	--	-------------

Severity Levels

Critical

Issues that can completely compromise protocol integrity or allow direct theft/loss of funds. Must be fixed before deployment.

High

Exploitable vulnerabilities affecting critical functionality, potentially causing loss of assets, broken pools, or bypassed core logic. Must be addressed immediately.

Medium

Weaknesses causing errors, inconsistent behavior, or revenue loss under certain conditions. Should be fixed but not catastrophic.

Low

Minor issues with limited impact, such as edge-case bugs or inefficient gas usage. Worth fixing for reliability.

Informational

Cosmetic or documentation issues. Do not pose direct threats but may confuse users or developers.

Issue Status

Security vulnerabilities identified that must be resolved and are currently unresolved.	Security vulnerabilities that have been identified and successfully fixed or mitigated.
Acknowledged	Partially Resolved
Vulnerabilities which have been acknowledged but are yet to be resolved.	Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.

Number of Security Issues per Severity

Severity	Count	Percentage	Status
HIGH	1	13.04%	Fixed
MEDIUM	1	4.35%	Fixed
LOW	1	30.43%	Fixed
INFORMATIONAL	2	30.43%	Fixed
GAS	0	-	-

Detailed Findings

[H-01] Inconsistent Timestamp Comparison Between Fuel TAI64 and Oracle UNIX Formats

Description The get_price function performs a staleness check using the condition

```
fn set_debug_timestamp(timestamp: u64) {
    only_owner();
    // Allow setting a custom timestamp for testing, but only in debug mode
    require(DEBUG, OracleError::DebugNotEnabled);
    @>      storage.debug_timestamp.write(timestamp);
}
#[storage(read, write)]
fn get_price() -> u64 {
    // Determine the current timestamp based on debug mode
    let current_time = match DEBUG {
        true => storage.debug_timestamp.read(),
    @>      false => timestamp(),
    };
    @>      if current_time <= stork_timestamp + TIMEOUT {}
    @>      if current_time <= pyth_price.publish_time + TIMEOUT {}
    @>      if current_time <= redstone_timestamp + TIMEOUT {}
```

Here, `current_time` is obtained from the Fuel networks block timestamp (`timestamp()`), which is expressed in TAI64 format. In contrast, oracle-provided timestamps (from Pyth, Redstone, and Stork) use the UNIX timestamp format (seconds since 1970).

Since TAI64 and UNIX timestamps are fundamentally different representations, comparing them directly will yield meaningless or incorrect results. This renders the staleness validation ineffective, potentially causing stale oracle data to be accepted as fresh, or fresh data to be rejected as stale.

Impact

This issue would cause inconsistent behaviour and incorrect staleness check.

Recommended mitigation

We recommend using the Fuel SDK [Date conversion](#) utilities to handle timestamp conversions. Alternatively, you may implement the necessary conversions manually, ensuring both TAI64 and UNIX timestamps are properly aligned.

[M-01] Recursive `transfer_ownership` Call Causes Guaranteed Revert

Description The contract defines a public function `transfer_ownership` which is intended to delegate ownership transfer to the `sway_libs::ownership::transfer_ownership` function. However, because both share the same name, the locally defined function shadows the imported one.

Sway allows defining a local function with the same name as an imported one, but if the call `transfer_ownership(new_owner)` is not disambiguated to explicitly reference the imported function, it resolves to the local function itself. This causes infinite recursion, eventually consuming all available gas and reverting the transaction

```
use sway_libs::ownership::{
    _owner,
    initialize_ownership,
    transfer_ownership,
    only_owner,
};
#[storage(read, write)]
@> fn transfer_ownership(new_owner: Identity) {
    @> transfer_ownership(new_owner);
}
```

Impact:

This may result in ownership being permanently untransferable.

Recommended mitigation

Either disambiguate the call to explicitly reference the imported function:

```
fn transfer_ownership(new_owner: Identity) {
    - transfer_ownership(new_owner);
    + ownership::transfer_ownership(new_owner);
}
```

Or rename the contract function to avoid shadowing:

```
- fn transfer_ownership(new_owner: Identity) {
+ fn transfer_owner(new_owner: Identity) {
```



```
transfer_ownership(new_owner);  
}
```

[L-01] `get_price` Uses Pyth Price Without Validating Confidence Interval

Description Within the `get_price` function, when Stork is either not configured or its price is stale, the contract falls back to fetching a price from Pyth. After fetching, the implementation only checks staleness via:

```
if current_time <= pyth_price.publish_time + TIMEOUT {  
  _set_last_good_price(last_good_price, pyth_price.price, pyth_price.publish_time);  
  return pyth_price.price;  
}
```

However, Pyth explicitly recommends validating the confidence interval alongside the reported price. The confidence interval indicates the uncertainty of the price, particularly under volatile or illiquid market conditions. Relying solely on the publish time while ignoring confidence may lead to using unreliable or manipulated prices.

Best practice when integrating with Pyth feeds is to reject prices with excessively wide confidence intervals, as these may reflect temporary dislocations, low liquidity, or abnormal market events.

Impact

The protocol may accept Pyth prices that are recent but inaccurate or untrustworthy.

Recommended mitigation

Incorporate a confidence interval check when validating Pyth prices similar to Fluid

```
fn is_pyth_price_stale_or_outside_confidence(pyth_price: Price, current_time: u64) -> bool  
{  
  let confidence_threshold = pyth_price.price / 25;  
  return current_time > pyth_price.publish_time + TIMEOUT || pyth_price.confidence >  
    confidence_threshold;  
}
```

[I-01] Missing Zero Address Validation in `transfer_ownership`

Description The `transfer_ownership` function currently forwards ownership transfer calls without checking whether the `new_owner` is a valid non-zero address. This means ownership can be assigned to the zero address, effectively renouncing ownership of the contract.

While renouncing ownership may be intentional in some cases, the `sway_libs::ownership` library already provides a dedicated `renounce_ownership` function for this purpose. Relying on `transfer_ownership` to achieve the same result is ambiguous and may create confusion.

[I-02] `get_price` Always Prioritizes Stork Oracle, Ignoring Potentially Fresher Data

Description The `get_price` function implements a prioritization order when fetching asset prices. Currently, if Stork is configured and its price is not considered stale (`current_time <= stork_timestamp + TIMEOUT`), the function immediately returns the Stork price and does not evaluate data from Pyth or Redstone.

```
if current_time <= stork_timestamp + TIMEOUT {  
  _set_last_good_price(last_good_price, stork_price, stork_timestamp);  
  return stork_price; // Early exit  
}
```

This logic means that as long as Stork provides a price within the staleness window, it is treated as authoritative, regardless of whether Pyth or Redstone have fresher timestamps or more reliable/confident data.

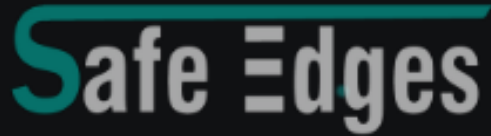
While the comments in the contract suggest that Stork is intended to be the exclusive oracle for stFuel, the current implementation applies this preference broadly. This undermines the redundancy benefits of integrating multiple oracle providers.

Impact

As a result of this prioritization, `get_price` can potentially return less up-to-date prices and also introduces a single-point-of-failure risk

Recommended mitigation

Stork can be prioritized for stFuel or other designated assets; otherwise, refactor `get_price` to consider all configured oracle prices



REVOLUTIONIZING BLOCKCHAIN AUDITS

Founded in 2023, Safe Edges is the largest blockchain security company that ensures the safety and reliability of smart contracts and blockchain-based protocols. Through the utilization of our world-class technical expertise, alongside our proprietary, innovative technology, we support the success of our clients with best-in-class security, all whilst realizing our overarching vision: ensuring blockchain remains secure and trustworthy.

THANK YOU FOR WORKING WITH US



SAFEEDGES.IN